

Student 16 **MALAVIKAH V** 192419061RC

10 / 10 submitted

Q1. Selection Sort

Score: 10.00 TC: 3/3 passed

CODE

```
arr = list(map(int, input().split()))

for i in range(len(arr)):
    min_idx = i
    for j in range(i + 1, len(arr)):
        if arr[j] < arr[min_idx]:
            min_idx = j
    arr[i], arr[min_idx] = arr[min_idx], arr[i]

print(arr)
```

INPUT

23 78 45 8 32 56

OUTPUT

[8, 23, 32, 45, 56, 78]

Q2. Insertion Sort

Score: 10.00 TC: 3/3 passed

CODE

```
arr = list(map(int, input().split()))

for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1
    while j ≥ 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1
    arr[j + 1] = key

print(arr)
```

INPUT

8 5 7 1 9 3

OUTPUT

[1, 3, 5, 7, 8, 9]

Q3. Merge Sort

Score: 10.00 TC: 3/3 passed

CODE

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left_half = arr[:mid]
        right_half = arr[mid:]

        merge_sort(left_half)
        merge_sort(right_half)

    i = j = k = 0

    while i < len(left_half) and j < len(right_half):
        if left_half[i] < right_half[j]:
            arr[k] = left_half[i]
            i += 1
        else:
            arr[k] = right_half[j]
            j += 1
        k += 1

    while i < len(left_half):
        arr[k] = left_half[i]
        i += 1
        k += 1

    while j < len(right_half):
        arr[k] = right_half[j]
        j += 1
        k += 1

arr = list(map(int, input().split()))

merge_sort(arr)

print(arr)
```

INPUT

5 2 9 1

OUTPUT

[1, 2, 5, 9]

SIMATS Engineering • CSE • CSA0802 — Python Programming • 16-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q4. Diagonal Matrix

Score: 10.00 TC: 3/3 passed

CODE

```
import re

def is_diagonal_matrix(matrix):
    rows = len(matrix)
    cols = len(matrix[0])

    if rows != cols:
        return "Not Diagonal Matrix"

    for i in range(rows):
        for j in range(cols):
            if i != j and matrix[i][j] != 0:
                return "Not Diagonal Matrix"

    return "Diagonal Matrix"

s = input().strip()
rows_raw = re.findall(r'\[(.*?)\]', s[1:-1])

matrix = []
for row_str in rows_raw:
    matrix.append([int(x) for x in row_str.split(',')])

print(is_diagonal_matrix(matrix))
```

INPUT

[[1,2],[0,3]]

OUTPUT

Not Diagonal Matrix

Q5. Row and Column Sums

Score: 6.67 TC: 2/3 passed

CODE

```
s = input().strip()

clean_s = s.strip()[2:-2]

row_strings = clean_s.split('],[')

matrix = []
for r in row_strings:
    matrix.append([int(x.strip()) for x in r.split(',') if x.strip()])

row_sums = [sum(row) for row in matrix]

num_cols = len(matrix[0]) if matrix else 0
col_sums = [sum(matrix[r][c] for r in range(len(matrix))) for c in range(num_cols)]

print(f"Row= {row_sums}")
print(f"Col= {col_sums}")
```

INPUT

[[2,3,4],[1,5,6]]

OUTPUTRow= [9, 12]
Col= [3, 8, 10]

Q6. Maximum Subarray Sum

Score: 6.67 TC: 2/3 passed

CODE

```
arr = list(map(int, input().split()))

max_so_far = arr[0]
curr_max = arr[0]

for i in range(1, len(arr)):
    curr_max = max(arr[i], curr_max + arr[i])
    max_so_far = max(max_so_far, curr_max)

print(max_so_far)
```

INPUT

-2 1 -3 4 -1 2 1 -5 4

OUTPUT

6

Q7. Common Elements Among Three Lists

Score: 10.00 TC: 3/3 passed

CODE

```
input_str = input().strip()
lists = input_str.split(';')

set1 = set(map(int, lists[0].split()))
set2 = set(map(int, lists[1].split()))
set3 = set(map(int, lists[2].split()))

common = set1.intersection(set2, set3)

print(sorted(list(common)))
```

INPUT

5 6;6 7;6 8

OUTPUT

[6]

Q8. Matrix Multiplication

Score: 6.67 TC: 2/3 passed

CODE

```
import re

input_str = input().strip()

parts = re.findall(r'[AB]=\[.*?\s*\](?=\s*[AB]=\[.*?\s*\])', input_str)

def parse_matrix(matrix_str):
    rows_raw = re.findall(r'\[.*?\]', matrix_str)
    matrix = []
    for r in rows_raw:
        if r.strip():
            matrix.append([int(x.strip()) for x in r.split(',') if x.strip()])
    return matrix

A_str, B_str = parts[0], parts[1]
A = parse_matrix(A_str)
B = parse_matrix(B_str)

rows_A = len(A)
cols_A = len(A[0])
cols_B = len(B[0])

result = [[0] * cols_B for _ in range(rows_A)]

for i in range(rows_A):
    for j in range(cols_B):
        for k in range(cols_A):
            result[i][j] += A[i][k] * B[k][j]

if len(result) == 1:
    print(result)
else:
    print("[", end="")
    for i in range(len(result)):
        if i == 0:
            print(f"{result[i]}")
        elif i == len(result) - 1:
            print(f" {result[i]}")
        else:
            print(f" {result[i]}")
```

INPUT

A=[[2]] B=[[3]]

OUTPUT

[[6]]

Q9. Sort Rotate Min Max

Score: 3.33 TC: 1/3 passed

CODE

```
import re

input_str = input().strip()

list_match = re.search(r'List=([0-9\s-]+)', input_str)
k_match = re.search(r'k=(\d+)', input_str)

arr = list(map(int, list_match.group(1).split()))
k = int(k_match.group(1))

sorted_arr = sorted(arr)

n = len(sorted_arr)
k = k % n if n > 0 else 0
rotated_arr = sorted_arr[k:] + sorted_arr[:k]

min_val = min(sorted_arr)
max_val = max(sorted_arr)

print(f"Sorted= {sorted_arr}")
print(f"Rotated= {rotated_arr}")
print(f"Min= {min_val}")
print(f"Max= {max_val}")
```

INPUT

List=5 2 8 1 4,k=2

OUTPUT

```
Sorted= [1, 2, 4, 5, 8]
Rotated= [4, 5, 8, 1, 2]
Min= 1
Max= 8
```

Q10. Sort Matrix Search

Score: 3.33 TC: 1/3 passed

CODE

```
import re

input_str = input().strip()

find_match = re.search(r'Find=(-?\d+)', input_str)
target = int(find_match.group(1))

matrix_match = re.search(r'Matrix=\[(.*)\]', input_str)
matrix_str = matrix_match.group(1)

rows_raw = re.findall(r'\[[0-9\s,-]+\]', matrix_str)

matrix = []
for r in rows_raw:
    if r.strip():
        matrix.append([int(x.strip()) for x in r.split(',') if x.strip()])

flat = []
for row in matrix:
    flat.extend(row)
flat.sort()

rows = len(matrix)
cols = len(matrix[0]) if rows > 0 else 0

sorted_matrix = []
for i in range(rows):
    sorted_matrix.append(flat[i * cols : (i + 1) * cols])

pos = None
for r in range(rows):
    for c in range(cols):
        if sorted_matrix[r][c] == target:
            pos = (r, c)
            break
    if pos is not None:
        break

if pos is not None:
    print(f"Sorted= {sorted_matrix}")
    print(f"Position= {pos}")
else:
    print("Not Found")
```

INPUT

Matrix=[[4,2],[8,6]], Find=5

OUTPUT

Not Found